

Obtain an X.509 Certificate

- Requires both a Public (.crt ,.cer) and Private Key (.p12,.pfx).
- See script below for certificate requirements.
- Edit the DNSName and Subject
- Edit the NotBefore and NotAfter values
- The .CER and .PFX file will be in your Documents folder.
- We create the certificate in the CurrentUser store due to permissions issues. It gets exported and then imported to the LocalMachine store. If we are testing this script and not running it from Intune (in the SYSTEM context), we need to grant ourselves access to it.

```
$params = @{
    Type = 'Custom'
    DnsName = 'ianbaxter.onmicrosoft.com'
    CertStoreLocation = 'Cert:\CurrentUser\My'
    KeyExportPolicy = 'Exportable'
    KeySpec = 'Signature'
    Subject = 'CN=Data Collect'
    KeyUsage = "DigitalSignature"
    KeyAlgorithm = 'RSA'
    KeyLength = 2048
    HashAlgorithm = 'SHA256'
    NotBefore = (Get-Date)
    NotAfter = (Get-Date).AddMonths(24)
}

#
# Create the Self-Signed Certificate
New-SelfSignedCertificate @params
# Locate and load the certificate we just created
$cert = Get-ChildItem -Path "Cert:\CurrentUser\My" | Where-Object {$_.Subject -Match "Data Collect"}
# Get thumbprint
$thumbprint = ($cert | Select-Object Thumbprint, FriendlyName).Thumbprint
#
# Export the Public Key (.CER)
Export-Certificate -Cert $cert -FilePath "$env:USERPROFILE\Documents\Data_Collect.cer"
#
# Create the password we need for the Private Key
# Enter password here
$MyPassword = "SomePassword"
$mypwd = ConvertTo-SecureString -String $MyPassword -Force -AsPlainText
#
# Export the Private Key (.PFX)
Export-PfxCertificate -Cert $cert -FilePath "$env:USERPROFILE\Documents\Data_Collect.pfx" -Password $mypwd
#
# Delete by ThumbPrint
Remove-Item -Path "Cert:\CurrentUser\My\$thumbprint" -DeleteKey
```

Create and secure an App Registration

Launch the Entra ID portal

Choose “App Registrations” from the left menu bar.

Click “New Registration” from the top menu bar.

Give your Service Principal a name.

Unless you know you need it, you should probably just allow “Single Tenant Only” for the Account Types.

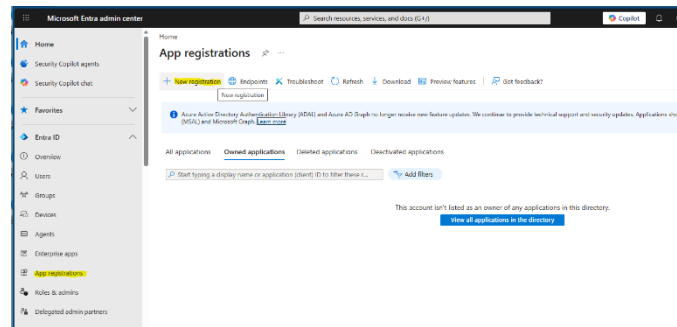
You should make a note of the following properties:

- Application (client) ID
- Directory (tenant) ID

These will be needed to test the Custom Compliance script.

Select “Certificates and Secrets” from the left menu bar.

Select the “Certificates” tab and click the “Upload Certificate” button.



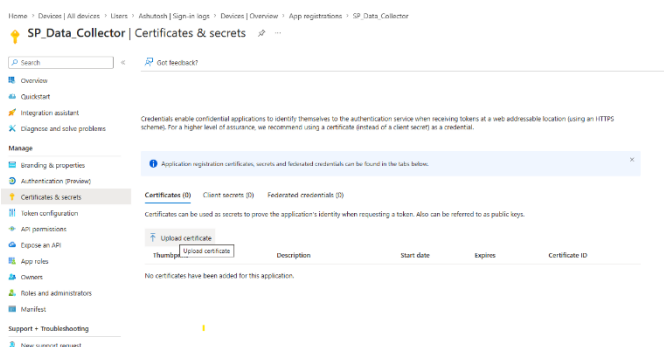
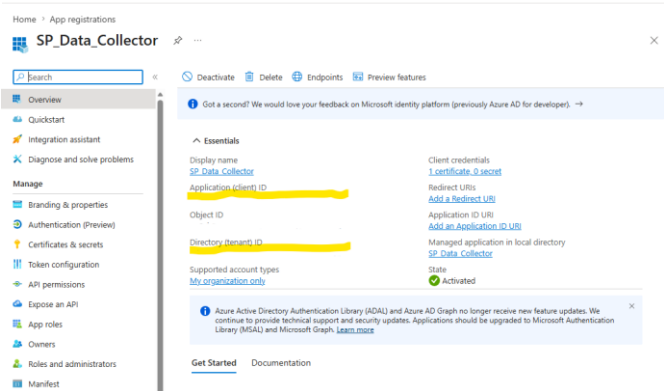
Register an application

* Name
The user-facing display name for this application (this can be changed later).

Supported account types
Choose the account types that can use this application or access this API.
 [Help me choose](#)

Redirect URI (optional)
We'll return the authentication response to this URI after successfully authenticating the user. Providing this now is optional and it can be changed later, but a value is required for most authentication scenarios.

Register an app you're working on here. Integrate gallery apps and other apps from outside your organization by adding from [Enterprise applications](#).

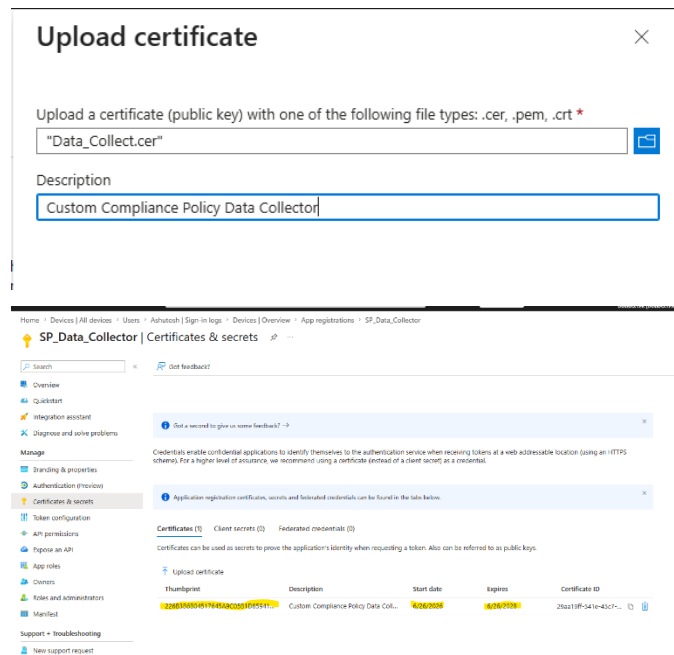


Select the file containing the public key (.crt or .cer).

Give the Descriptor some data to properly identify the certificate.

Make a note of the thumbprint so you can verify the private key when you install it on the test device.

Verify that you have the right dates on the certificate. (Note: You will see an icon to the right of the certificate if it has expired.)

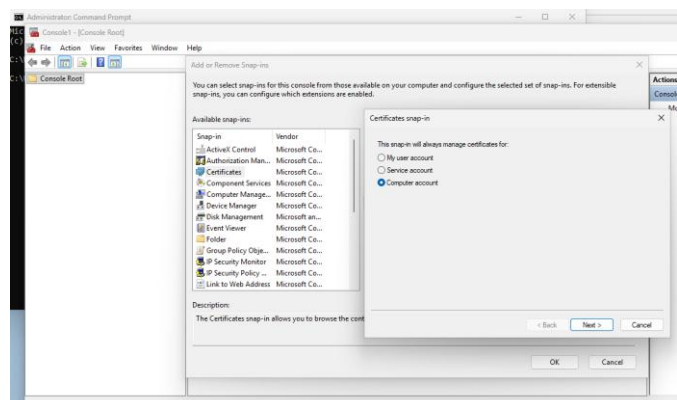


Install the Private Key on the test machine

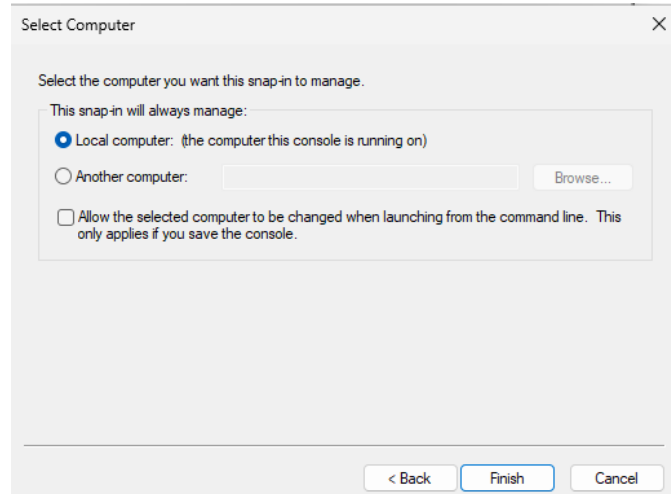
Since this is a test machine, you will be running the script as a user. The LocalMachine certificate store restricts users from reading the private key information from the certificate.

You need to have administrative rights to perform this step. Use the Windows search to find the icon for “CMD”. Right-click on the icon and choose “Run as Administrator”. Using the command prompt, run “MMC.EXE”, Select “File”, “Add/Remove Snapin...” and add the “Certificates” snapin.

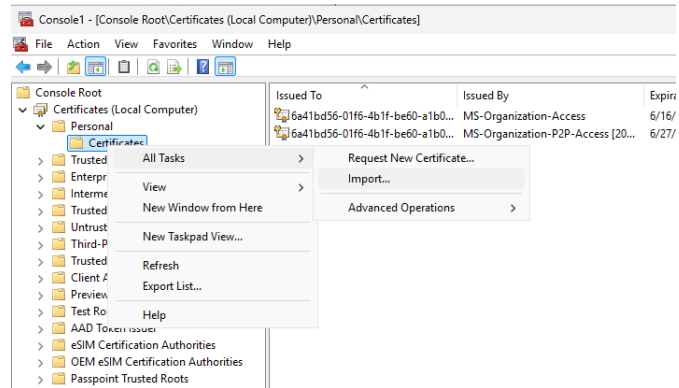
Select “Computer Account” and click “Next.”



Select “Local Computer” and click “Finish”.



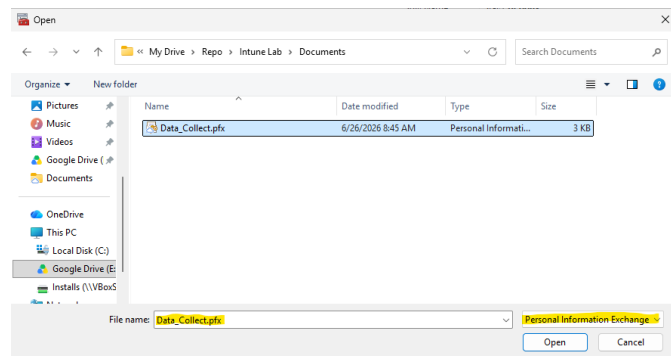
Locate the \Personal\Certificates folder in the left pane. Right-click, chose “All Tasks” and “Import...”



Set the file type dropdown to “Personal Information Exchange”.

Browse to your private key file and select it.

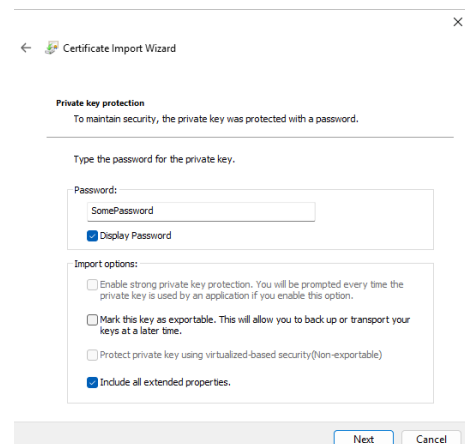
Click “Open”.



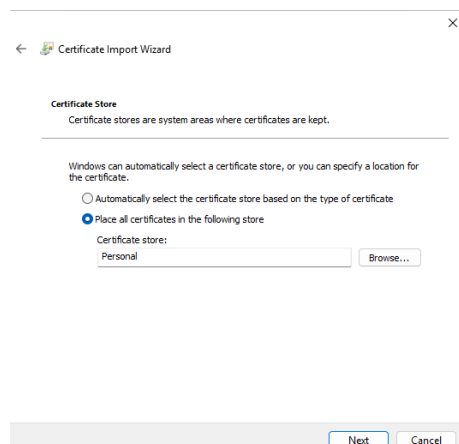
Remember the password you used to create the private key file? Enter it in the password text box.

If you don’t have the password, you need to get a new certificate and start over.

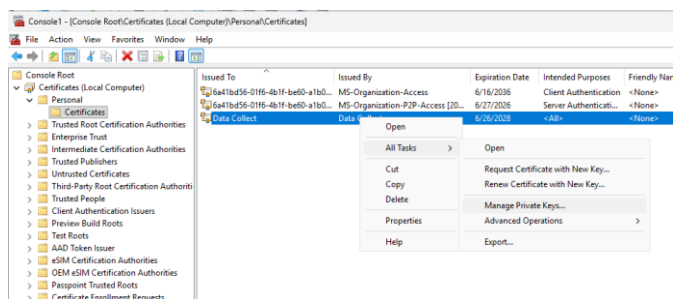
Do not select the option to make the certificate exportable.



Verify the certificate store is “Personal” and click “Next”.



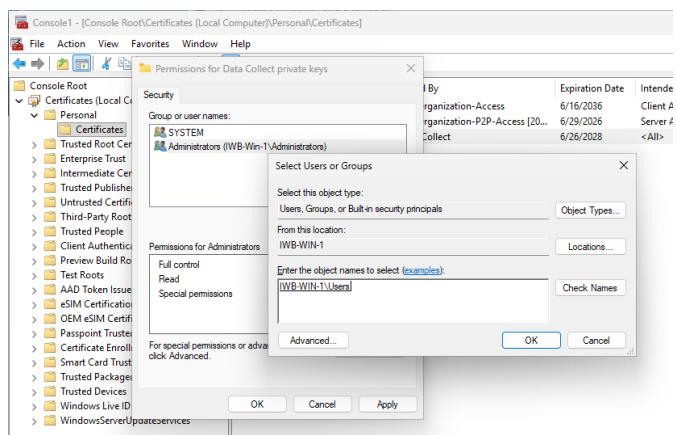
Right-click on the certificate, choose “All Tasks” and “Manage Private Keys...”.



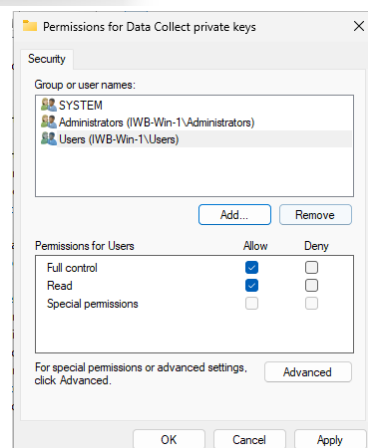
A permissions dialog will pop up. Click “Add”.

Enter “Users” in the bottom box, click “Check Names” to ensure the group or account can be resolved.

Click “OK”.



Verify the group or account has at least “Read” and “Full Control” access. You need at least “Read”, but “Full Control” will be needed to delete the certificate later.



Alter the Inventory Script and test (save JSON)

We are going to use a Windows PowerShell script for testing (Custom_Inventory.ps1). You will need to run the script using administrative access.

Enter “Powershell_ise” in the Windows Search box. Locate the icon for “Windows PowerShell ISE 5.1”, right-click on it and choose “Run as Administrator”.

You will need to run “Set-ExecutionPolicy -Scope LocalMachine -ExecutionPolicy Unrestricted” in the console.

The first few lines of the script are configuration variables.

Populate:

- \$ApplicationID
- \$TenantID
- \$CertName
- \$LogFile

```
7 # Service Principal Data (Required for Token)
8 $ApplicationID = "<Application/Client ID>"
9 $TenantID = "<Tenant ID>"
10 $CertName = "Data Collect"
11 #
12 # Log Analytics Data (for JSON submission)
13 $DCRImmutableId = "<DCR Immutable ID>"
14 $DCEURI = "<DCE Data Ingestion URL>"
15 $StreamName = "<Stream Name from DCR JSON>"
16 #
17 # Script logging
18 $LogFile = "<Log File Path>"
19 #
```

The next block of code are the functions that provide functionality to find the correct certificate and then use it to obtain an access token via OAuth2 after authenticating as the Service Principal.

At the beginning of the script, we use those functions. The script will not run unless a valid certificate is found.

```
157 #
158 # BEGIN SCRIPT
159 #
160 #
161 # Get the first valid certificate that matches $CertName
162 $Cert = Get-ValidCertificate -CertName $CertName
163 # If one was found
164 if ($Cert) {
165     # Get the thumbprint
166     $Thumbprint = (Get-Childitem -Path "Cert:\LocalMachine\My" | Where-Object {$_.Subject -Match $CertName}).Thumbprint
167     # BEGIN INVENTORY
168 }
```

Customize the Inventory block to collect the data you want to report via JSON upload. If you are reporting for both Windows and MacIntosh, remember that your JSONs must have identical structures and plan accordingly.

```
167 #
168 # BEGIN INVENTORY
169 #
170 #
171 #
172 # Get the ManagedDeviceID and Name from Intune enrollment
173 try {
174     # If the device is enrolled
175     if (@(Get-Childitem HKLM:SOFTWARE\Microsoft\Enrollments\ -Recurse |
176         # Get the source for the managed device information
```

Use the code example to create the structure we are going to export to JSON.

```
422 #
423 # Compile all the settings in a form we can easily convert to JSON
424 $Inventory = New-Object System.Object
425 $Inventory | Add-Member -MemberType NoteProperty -Name "ManagedDeviceName" -Value $ManagedDeviceName -Force
426 $Inventory | Add-Member -MemberType NoteProperty -Name "AzureADDeviceID" -Value $AzureADDeviceID -Force
427 $Inventory | Add-Member -MemberType NoteProperty -Name "ManagedDeviceID" -Value $ManagedDeviceID -Force
428 $Inventory | Add-Member -MemberType NoteProperty -Name "DefenderState" -Value $DefenderState -Force
429 $Inventory | Add-Member -MemberType NoteProperty -Name "DefenderStart" -Value $DefenderStart -Force
430 $Inventory | Add-Member -MemberType NoteProperty -Name "DefSpySigAge" -Value $Defender_SpySigAge -Force
431 $Inventory | Add-Member -MemberType NoteProperty -Name "DefMissSigAge" -Value $Defender_MissSigAge -Force
432 $Inventory | Add-Member -MemberType NoteProperty -Name "DefAVSigAge" -Value $Defender_AVSigAge -Force
433 $Inventory | Add-Member -MemberType NoteProperty -Name "DefWEngine" -Value $Defender_WEngine -Force
434 $Inventory | Add-Member -MemberType NoteProperty -Name "BitLockerState" -Value $BitLockerState -Force
435 $Inventory | Add-Member -MemberType NoteProperty -Name "BitLockerStart" -Value $BitLockerStart -Force
436 $Inventory | Add-Member -MemberType NoteProperty -Name "BitEncrypted" -Value $BitEncrypted -Force
437 $Inventory | Add-Member -MemberType NoteProperty -Name "BitEncryption" -Value $BitEncryption -Force
438 $Inventory | Add-Member -MemberType NoteProperty -Name "BitProtected" -Value $BitProtected -Force
439 $Inventory | Add-Member -MemberType NoteProperty -Name "BitProtector" -Value $BitProtector -Force
440 foreach ($thiskey in ($NetHash.Keys | Sort-Object)) {
441     $Name = "MAC" + $thiskey.ToString()
442     $Value = $NetHash[$thiskey]
443     $Inventory | Add-Member -MemberType NoteProperty -Name $Name -Value $Value -Force
444 }
445 #
446 # Convert to JSON for sending the data to Log Analytics Workspace
447 $body = $Inventory | ConvertTo-Json
448 #
449 # END INVENTORY
450 #
```

Set a breakpoint on this line of code.

If not, you need to fix something either with your certificate or with your inventory code.

Comment out the highlighted lines of code and remove the breakpoint.

Launch the Azure Portal, search for Log Analytics workspaces.

Click on “Create”.

You may need to create a Resource Group.
If so, click the “Create New” option.

Populate the Name and Region, click “Review + Create”.

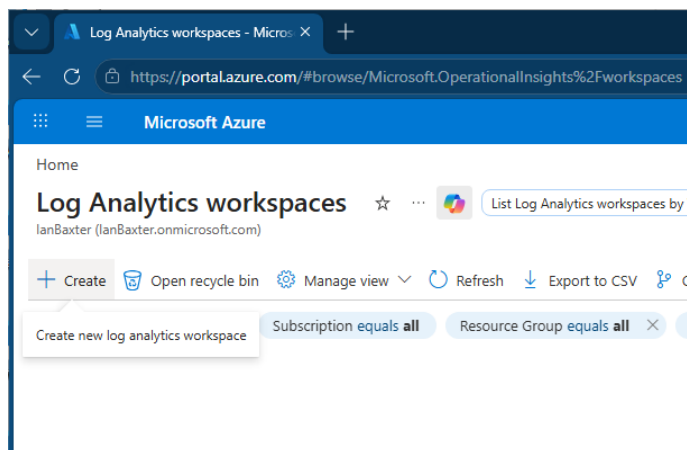
When the validation passes, click “Create”.

Click the “Go to Resource” option.

```

464 # END INVENTORY
465
466 #
467 # Uncomment for DEBUGGING
468 #
469 #
470 # Get the auth token
471 $bearerToken = Get-AuthTokenWithCert -TenantId $TenantID -ClientId $ApplicationID -CertThumbprint $thumbprint
472
473 # Uncomment for DEBUGGING
474 #bearerToken
475
476 # Don't save the whole token to the log, just enough to know we got it
477 $token = ($bearerToken -split " ")[1]
478 Write-Log -LogFile $LogFile -Message $Message
479
480 # Send the JSON to the Log Analytics Data Ingestion API

```

[illegible]

Project details

Select the subscription to manage deployed resources and costs. Use resource groups like folders to organize and manage all your resources.

Subscription * 



Resource group * 



[Create new](#)

A resource group is a container that holds related resources for an Azure solution.

Name * 



Instance details

Name * 

Region * 

[Review & Create](#) [< Previous](#) [Next > Tags](#)

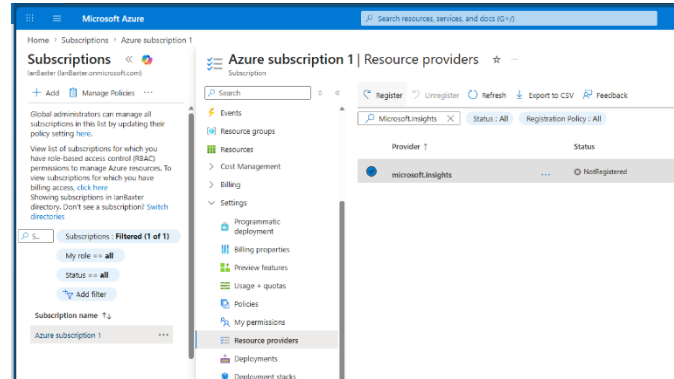
Register Microsoft Insights resource provider

In the Azure Portal, search for “Subscription” and load the applet.

Select the correct subscription. Click on “Resource Providers”, search for “Microsoft.Insights”.

Select it and click the “Register” option.

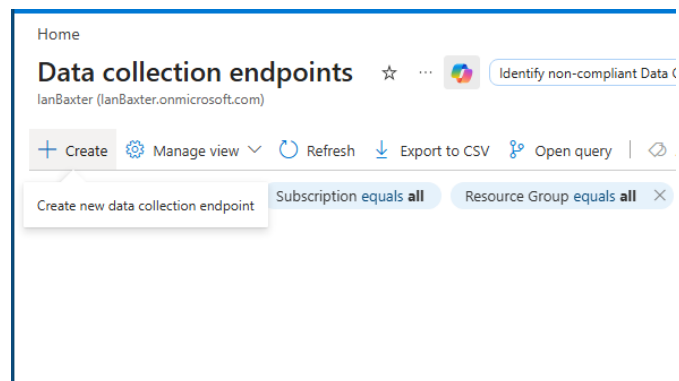
Wait for the notification that the registration has completed.



Create a DCE (Data Collection Endpoint)

Open the Azure Portal, search for “Data Collection Endpoints” and launch the applet.

Click the “Create” option.



Populate the “Endpoint Name”, select the appropriate Subscription, Resource Group and Region.

The Region should be the same as your Tenant and must match the values for the DCR and Log Analytics workspace.

Click “Review and Create.”

Create data collection endpoint

Basics Tags Review + create

Select the subscription to manage deployed resources and costs. Use resource groups like folders to organize and manage all of your resources. [Learn more](#)

Endpoint details

Endpoint Name *	Custom-Compliance ✓
Subscription * ⓘ	Azure subscription 1 ▼
Resource Group * ⓘ	Intune-Reporting ▼ Create new
Region * ⓘ	Canada Central ▼

[Review + create](#) < Previous Next : Tags >

Create the Log Analytics Table and DCR from JSON

Let me caution you here. You need to do this once; do it right and fully understand what you are doing at this step. It seems simple from all the Internet documentation, but it really is not and is poorly documented.

If you make a mistake, you cannot delete the old table and DCR. The data that underlies them can persist for up to 90 days. The data for the table or DCR does not get deleted and will re-populate in any new one you recreate. The only other solution is to create one with a different name.

With the log analytics workspace open, expand the “Settings” and select “Tables”.

Click the “Create” option.

The screenshot shows the Microsoft Log Analytics OMS interface. The breadcrumb navigation is 'Home > MicrosoftLogAnalyticsOMS | Overview > Custom-Compliance'. The main heading is 'Custom-Compliance | Tables' with a star icon. Below the heading is a search bar and a '+ Create' button. The left sidebar shows the navigation menu with 'Tables' selected. The main content area displays a table with 6 results, showing columns for 'Table name', 'Type', and 'Plan'.

Table name	Type	Plan
Alert	Azure table	Analytics
AppCenterError	Azure table	Analytics
ComputerGroup	Azure table	Analytics
InsightsMetrics	Azure table	Analytics
Operation	Azure table	Analytics
Usage	Azure table	Analytics

The table name must be the same as the table you will configure in the script with a few differences. The script uses a stream name which is “Custom-<TableName>_CL”. The “_CL” gets appended to the table name for “Custom Log” in Log Analytics, and the “Custom-” gets added by the DCR to form the data stream name.

Enter a description.

You should not have a DCR yet, so click the “Create a new data collection rule” option.

Home > MicrosoftLogAnalyticsOMS | Overview > Custom-Compliance | Tables

Create a custom log

Table details
Start by adding a name and description for the table you're creating. On the next step, upload a sample of your custom log and adjust the table details to your needs.

Table name * ✓
_CL

Description

Table plan

☒ Analytics ☐ Basic ☐ Auxiliary / Lake

[Learn more about the different table plans by clicking here.](#)

Data collection rule
Data collection rules (DCR) define the data coming into Azure Monitor and specify where that data should be sent or stored. [Learn more](#)

Data collection rule *

Data collection endpoint *

Choose your subscription and resource group, give the DCR a name.

Click “Done”.

Select the Data Collection Endpoint you created earlier, click “Next”.

Create a new data collection rule

Subscription *

Resource group *

[Create new](#)

Region
Canada Central

Name *

[Done](#) [Cancel](#)

Upload the Json data you saved from the script test.

You should see the warning that a transform has been created for TimeGenerated. If your JSON doesn't contain the mandatory TimeGenerated data, a transform will be automatically created for you.

You will also see all the names of values in the JSON you uploaded.

Click “Next” and “Continue”.

The screenshot shows the Microsoft Azure portal interface for creating a custom log. The top navigation bar includes the Microsoft Azure logo and the user's email address. The main heading is "Create a custom log". Below this, there are three tabs: "Basics", "Schema and transformation" (which is selected), and "Review". Under the "Schema and transformation" tab, there is a section for "Upload sample file" and a "Transformation editor" link. A large grey box with a cloud icon and text prompts the user to upload a sample of logs in JSON format. Below this, a warning message states: "There was no timestamp field found in the sample provided. The transformation was updated to populate TimeGenerated column with the timestamp of data ingestion. Please use the transformation editor to review or update the logic of TimeGenerated column population in the destination table. [Learn more](#)". At the bottom, a table displays the schema of the custom log with columns: TimeGenerated, ManagedDeviceName, AzureADDeviceID, ManagedDeviceID, DefenderState, and DefenderStart. The first row of data shows a timestamp, device name, and state.

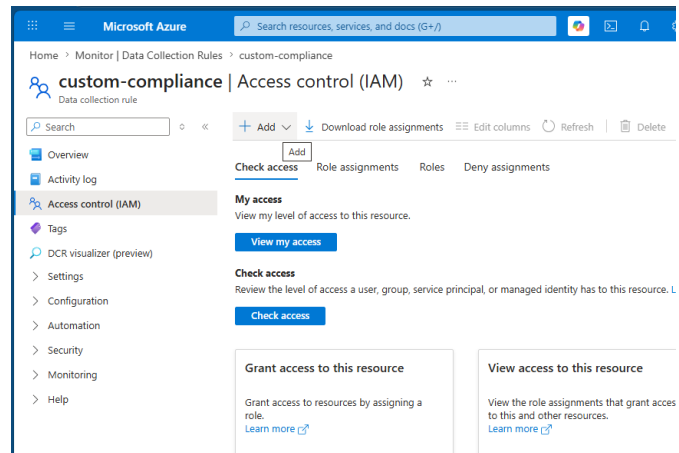
TimeGenerated	ManagedDeviceName	AzureADDeviceID	ManagedDeviceID	DefenderState	DefenderStart
2026-06-28T15:18:39...	IanBaxter_Windows_6/...	6a41bd56-01f6-4b1f-...	c5e23685-75af-4d76-...	Running	Automatic

Permission the Service Principal

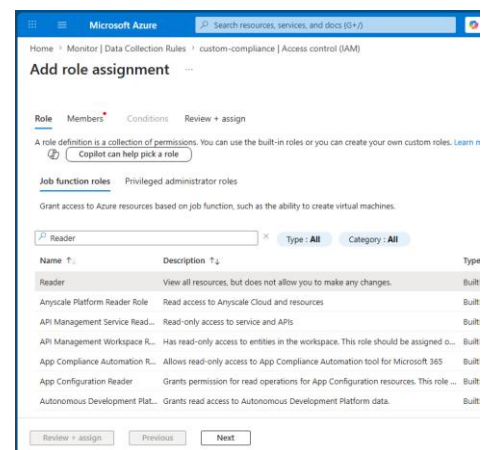
DCR

In the Azure Portal, search for “Data Collection Rule” and launch the applet.

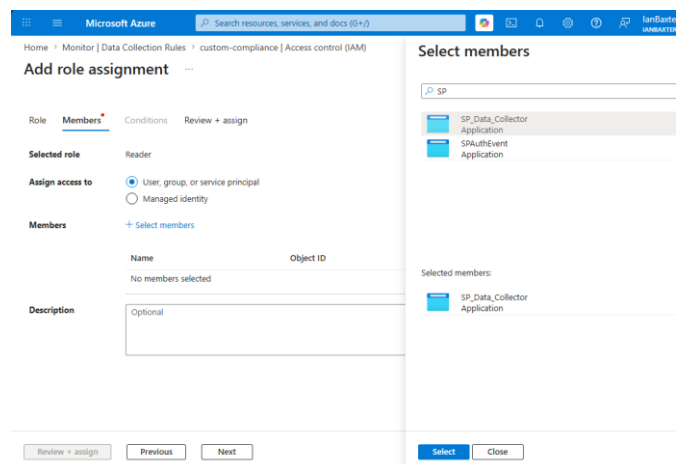
Select “Access Control” and click “Add”.
“Add Role Assignment”.



Search for and select the “Reader” role. Click “Next”.



Click “Add Members”, search for and select the Service Principal account and click the “Select” button.



Click “Review and Assign”.

Repeat these steps to add the “Monitoring Metrics Publisher” role.

Verify the roles have been properly assigned.

Workspace

Search “Log Analytics Workspaces” in the Portal, load the workspace you created.

Add the “Log Analytics Contributor” role to the Service Principal.

Home > Monitor | Data Collection Rules > custom-compliance | Access control (IAM)

Add role assignment

Role **Members** Conditions Review + assign

Selected role Reader

Assign access to ☒ User, group, or service principal ☐ Managed identity

Members + Select members

Name	Object ID	Type
SP_Data_Collector	fb2584b5-98ab-4d7f-86df-622f3dde45fb	App

Description Optional

Review + assign Previous Next

Microsoft Azure Search resources, services, and docs (G47)

Home > Monitor | Data Collection Rules > custom-compliance | Access control (IAM)

Search

Overview Activity log Access control (IAM) Tags DCR visualizer (preview) Settings Configuration Automation Security Monitoring Help

Number of role assignments for this subscription 3 4000

Search by name or email Type: All Role: All Scope: All scopes Group by: Role

All (3) Job function roles (2) Privileged administrator roles (1)

Name	Type	Role	Scope	Condition
Owner (1)				
Ian Baxter ianbaxter@ianba...	User	Owner	Subscription (inheri...	None
Reader (1)				
SP_Data_Collector f68f6252-a149-4...	Service principal	Reader	This resource	None
Monitoring Metrics Publisher (1)				
SP_Data_Collector f68f6252-a149-4...	Service principal	Monitoring Metrics Publ...	This resource	None

Showing 1 - 3 of 3 results.

Add or remove identities by pressing CTRL+SHIFT+V

Microsoft Azure Search resources, services, and docs (G47)

Home > Log Analytics workspaces > Custom-Compliance | Access control (IAM)

Search

Overview Activity log Access control (IAM) Tags Diagnose and solve problems Logs Resource visualizer Settings Tables Agents Usage and estimated costs Rules Network isolation Identity Linked storage accounts

Check access Role assignments Roles Deny assignments

Number of role assignments for this subscription 4 4000

Search by name or email Type: All Role: All Scope: All scopes Group by: Role

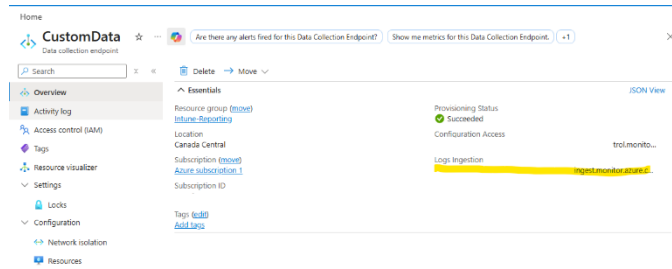
All (2) Job function roles (1) Privileged administrator roles (1)

Name	Type	Role	Scope	Condition
Owner (1)				
Ian Baxter ianbaxter@ianba...	User	Owner	Subscription (inheri...	None
Log Analytics Contributor (1)				
SP_Data_Collector f68f6252-a149-4...	Service principal	Log Analytics Contributor	This resource	None

Showing 1 - 2 of 2 results.

Update the script for the DCE and DCR and test

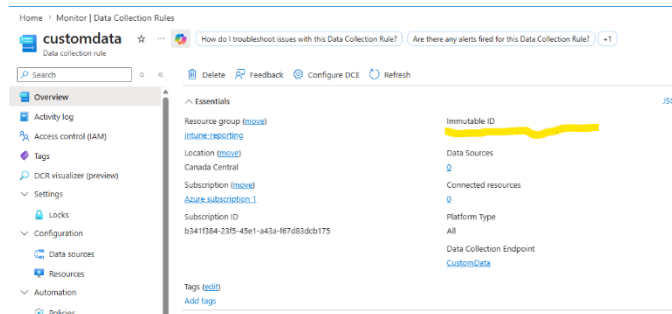
Copy the Logs Ingestion URL from the DCE Overview.



Paste the URL into the configuration block in the script.

```
11 #
12 # Log Analytics Data (for JSON submission)
13 $DcrImmutableId = "<DCR Immutable ID>"
14 $DceURI = "<DCE Data Ingestion URL>"
15 $streamName = "<Stream Name from DCR JSON>"
16 #
17 # Script loading
```

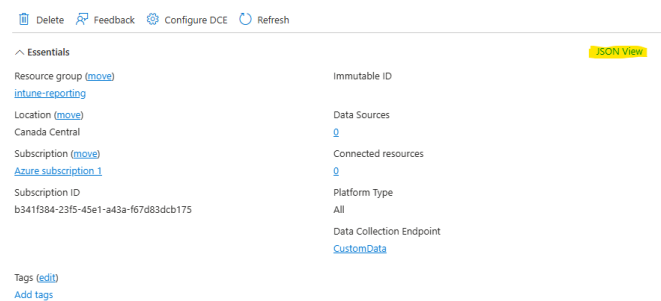
Copy the DCR Immutable ID from the DCR Overview.



Paste the ID into the configuration block of the script.

```
12 # Log Analytics Data (for JSON submission)
13 $DcrImmutableId = "<DCR Immutable ID>"
14 $DceURI = "<DCE Data Ingestion URL>"
15 $streamName = "<Stream Name from DCR JSON>"
16 #
```

In the DCR Overview, click the "JSON View" link.



Look for this section of the code and copy the stream name.

```
9 |
10 |
11 | "dataCollectionEndpointId": "/subscriptions/b341f384-23f5-45e1-a43a-f67d8
12 | "streamDeclarations": {
13 |   "Custom-CustomData-Ck": {
14 |     "columns": [
15 |       {
16 |         "name": "TimeGenerated",
17 |         "type": "datetime"
18 |       }
19 |     ]
20 |   }
21 | }
```

Paste the stream name into the correct line of the script.

Make sure you have a log file path specified. Given Intune's console ability to collect logs remotely, this can be very useful.

Besides the log, Diagnostic Settings in a DCR can also help with debugging.

Load the DCR, select "Diagnostic Settings" in the left menu and click "Add Diagnostic Setting..."

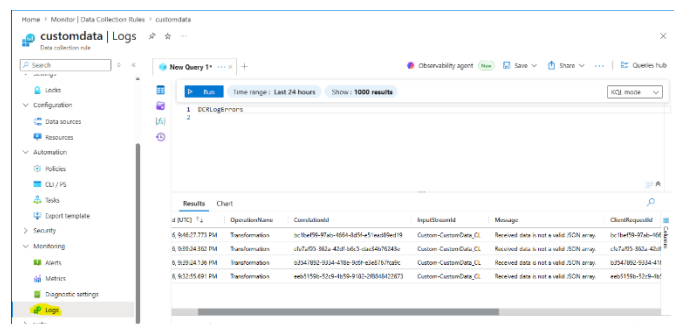
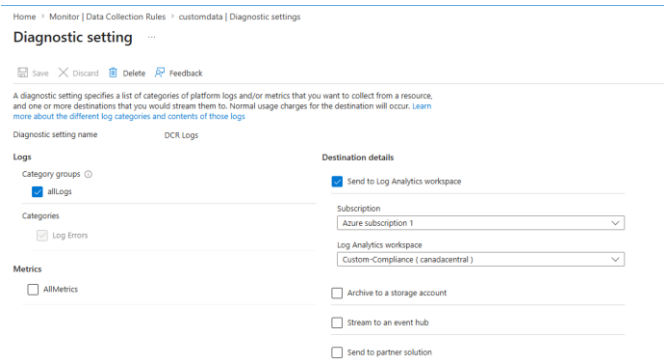
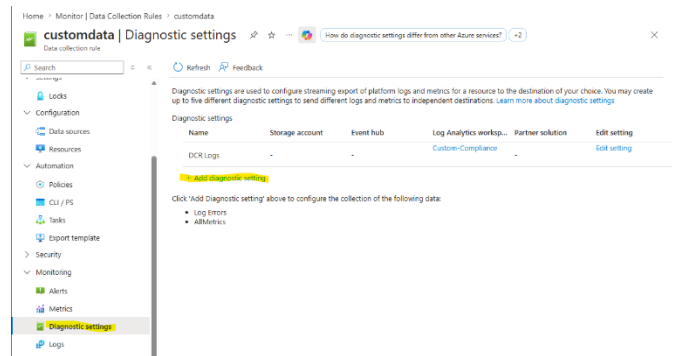
Configuring these options will send all the DCR logs to the Log Analytics workspace.

Selecting Logs in the left pane and clicking the "Run" button will show you any found issues.

If the configuration is correct, you can run the script and get a success message.

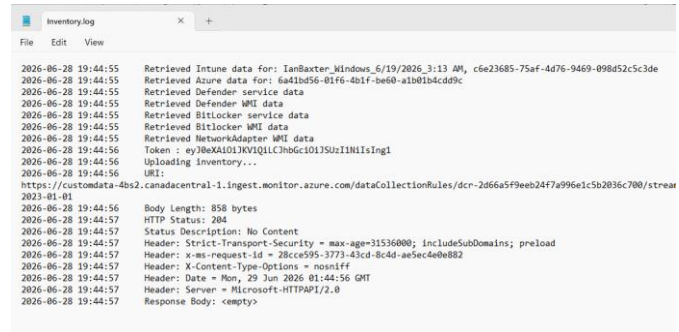
```
10 # Paste the stream name into the correct line of the script
11 #
12 # Log Analytics Data (for JSON submission)
13 $DcrImmutableId = "<DCR Immutable ID>"
14 $DceURI = "<DCE Data Ingestion URL>"
15 $streamName = "<Stream Name from DCR JSON>"
16 #
17 # Script logging
```

```
15 $streamName = "<Stream Name from DCR JSON>"
16 #
17 # Script logging
18 $LogFile = "<Log File Path>"
19 #
20 # If a log file has been specified and it already exists, d
21 if (-not [string]::IsNullOrEmpty($LogFile) -and (Test-
```

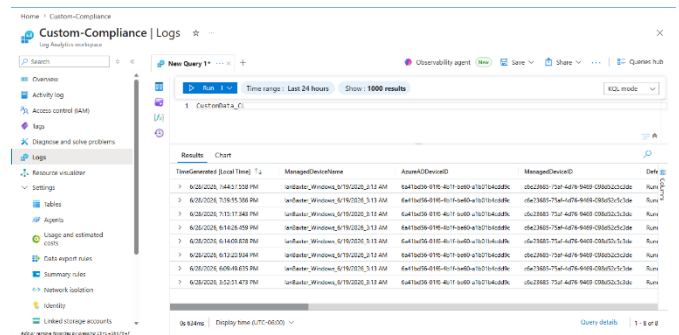


```
PS C:\WINDOWS\system32> E:\My Drive\Repo\Intune Lab\Projects\Custom Reporting\SP\Custom_Inventory.ps1
InventoryDate:28-06-19:39 DeviceInventory:OK (204)
PS C:\WINDOWS\system32> |
```

If not, the logs should show you where you went wrong.



If it went right, the data can be viewed in the Log Analytics table.



Remove the manually deployed Certificate

Deploy the Private Key to all devices

Create the Custom Compliance Policy and Deploy

Verify Data is being collected